CSE 151B/251B Final Report: Improving Mathematical Reasoning in Qwen3-4B

Team [BeijingBois]
University of California, San Diego

Abstract

We study the CSE 151B competition on mathematical reasoning, in which a fixed open-weight model, Qwen3-4B, must answer a set of math problems ranging from high-school to graduate level. The competition rules prohibit the use of external APIs, tools, and code interpreters at inference time, so all gains must come from intrinsic improvements to the model: prompt engineering, supervised fine-tuning (SFT) with parameter-efficient adapters, reinforcement learning with verifiable rewards, and self-reflection. Starting from the starter pipeline's baseline of **61.90%** on a held-out 127-question (roughly 85-15 training-validation split) validation set (79.47% MCQ, 53.13% free-form), we designed two format-aware system prompts targeting specific failure modes (grader-format mismatches, runaway reasoning, options-anchoring) that raise accuracy on the same set to **67.37%** (82.62% MCQ, 59.91% free-form). We then attempted to bake these gains into the weights via LoRA fine-tuning on reasoning traces distilled from a larger teacher model (Qwen3-30B-A3B-Thinking-2507) using rejection sampling. The fine-tune *underperformed* the prompt-only system, dropping to 64.29% overall; our diagnosis is that the corpus was too small ($\sim$950 problems) and the teacher traces were shorter than the base model's natural reasoning, which trained the model to under-reason. We therefore shipped the base model with format-aware prompting as our final submission, finishing **12th** on the final leaderboard — a climb of **36 positions** relative to our original public-leaderboard standing. We present the negative fine-tuning result, the data-quality issues we uncovered along the way, and the lessons we drew from them. Our code is available at https://github.com/YOUR-REPO-LINK-HERE.

1 Introduction

Problem Definition: The deep learning task here is to take a math problem written in plain text and have a language model output the right answer, where "the right answer" gets graded automatically. The model we have to work with is fixed — it's Qwen3-4B, a 4-billion-parameter open-source model — so we can't swap it out for something bigger. Each problem in the dataset comes with a question, a correct answer (or list of answers), and sometimes a set of multiple-choice options. The questions themselves come in a few different shapes: some ask for a single numerical or symbolic answer, some have multiple blanks the model has to fill in, and some are multiple-choice where the model just has to pick a letter. After the model writes out its response, a parser pulls the answer out of whatever it produced, and a judge checks whether that answer matches the ground truth. Our score on the leaderboard is just the fraction of problems we get right on a held-out test set whose answers we never see.

Problem Significance: Mathematical reasoning is one of the cleanest evaluation testbeds available for language models since answers are automatically verifiable by a symbolic judge, so reasoning improvements can be measured without subjective human grading or rater bias, and a model that

Preprint. Under review.

simply memorized training problems will fail on novel variants. This makes math a uniquely diagnostic setting for studying whether a model is actually reasoning rather than pattern-matching, and the techniques that move the needle here — chain-of-thought prompting, self-consistency, and RL with verifiable rewards — have historically transferred to other reasoning-heavy tasks like program synthesis, scientific question answering, and formal verification. On the practical side, a small math model that runs locally lets students use it as a free tutor on their own laptop, works in schools or countries where cloud APIs are blocked or unaffordable, helps researchers do quick symbolic checks without sending work to a third-party server, and stays usable in low-connectivity settings.

Technical Challenge: The biggest technical challenges for us were on the implementation and data sides. Early on, getting `bitsandbytes` to work was the main blocker — we ran into CUDA and driver compatibility issues, and we ultimately sidestepped them by loading the model in `bfloat16`. When we later moved to LoRA fine-tuning, we also had to work through several version-specific quirks in the training stack (e.g., API changes in TRL and `transformers`). The model itself isn't small: Qwen3-4B has a built-in "thinking" mode that wraps its reasoning in `<think>...</think>` tags, and running it over the full evaluation set takes hours, so iteration speed was a persistent bottleneck; we upgraded to Colab Pro to use A100 GPUs. Training cost was a challenge of its own: LoRA fine-tuning a 4B model with reasoning traces up to 16k tokens long on a single GPU is slow, and on our limited compute budget each run consumed a large fraction of a Colab session, which meant we could only afford a handful of training runs total and every mistake (a bad dataset version, a misconfigured run) cost us real time. Building the training data was similarly expensive — because we generated our own corpus of reasoning traces with a larger teacher model and rejection-sampled them against the judge, every iteration on the dataset meant hours of additional generation time before training could even start. Finally, the hardest and least anticipated challenge was *training-data quality*: our distilled corpus went through several broken versions (malformed thinking tags, answer leakage into prompts, train/inference prompt mismatches) before we had something trainable, and even the final clean version turned out to hurt performance for subtler reasons we discuss in Section 4.4.

Contributions: The starter code provides a minimal inference pipeline that loads Qwen3-4B-Thinking-2507 through vLLM, applies the chat template to each question with a brief generic system prompt, generates a single response, extracts the contents of the last `\boxed{}` group, and writes a CSV submission. Its core limitation is that the system prompt is generic — it tells the model where to put the answer but not what form it should take. Because the competition's grader does literal comparison, this leaves a meaningful fraction of mathematically correct answers scoring zero on surface-form mismatches alone (e.g. `1.363` versus `atan(4.76)`, or `0.625` versus `5/8`). The starter pipeline also makes no use of question type, despite the dataset containing three distinct formats (free-form single, free-form multi, multiple-choice) with very different answer conventions.

To evaluate the baseline and all of our changes, we held out a 127-question validation set from the public data, sampled to preserve the MCQ/free-form ratio of the full set. All validation numbers reported below come from this set, scored with the competition's symbolic judge.

Our main contributions are:

- **Reproducible baseline with per-type diagnostics:** We reproduced the starter pipeline end-to-end on Google Colab, reaching **61.90%** overall (79.47% MCQ, 53.13% free-form). The 26-point gap between MCQ and free-form pointed directly at free-form answer formatting as the largest source of recoverable losses.
- **Format-aware prompts targeting grader-alignment failures:** We designed two system prompts — one for free-form and one for multiple-choice — that explicitly enforce grader-aligned output formats, constrain the reasoning style to prevent runaway self-doubt loops, and (for MCQ) enforce a solve-then-match procedure to mitigate options-anchoring. This raises validation accuracy to **67.37%** (82.62% MCQ, 59.91% free-form), with the gain concentrated on free-form ($\sim$+6.8 points), and was the configuration behind our final submission, which placed **12th** on the final leaderboard, up **36 positions** from our original public standing.
- **A diagnosed negative result on parameter-efficient fine-tuning:** We built a full LoRA SFT pipeline — teacher-model data generation, rejection sampling, completion-only loss masking, training, and adapter-based vLLM inference — and found that fine-tuning *reduced* validation accuracy from 67.37% to 64.29%. We trace the degradation to a small corpus and

to teacher traces shorter than the base model's natural reasoning length, which suppressed the model's thinking. Identifying this failure mode is what let us make the right final-submission decision: ship the base model with format-aware prompting rather than a worse fine-tune.

2 Related Work

Chain of Thought Reasoning Chain-of-thought prompting, introduced by Wei et al. [4], elicits intermediate reasoning steps from large language models and substantially improves performance on reasoning tasks. Kojima et al. [3] showed that even a simple zero-shot instruction such as "Let's think step by step" is sufficient to trigger this behavior without exemplars. Our approach builds on this idea: we prompt the model to reason through the problem step by step, while explicitly instructing it to keep its reasoning concise and avoid overthinking.

LoRA Finetuning Low-Rank Adaptation (LoRA), proposed by Hu et al. [2], enables parameter-efficient finetuning by freezing pretrained weights and injecting trainable low-rank matrices into transformer layers. A common application is reasoning distillation, where a smaller student model is finetuned on chain-of-thought traces generated by a stronger teacher [1]. Following this paradigm, we performed LoRA finetuning on reasoning traces generated by Qwen3-30B-A3B-Thinking-2507, although in our setting this degraded performance relative to prompt engineering alone.

3 Methods

Problem Setting: Let $\mathcal{D} = \{(x_i, y_i, t_i, o_i)\}_{i=1}^{N}$ denote the dataset, where x_i is the question text, y_i is the ground-truth answer (a list of strings or a single letter), $t_i \in \{\text{NV, EX, EQ, INT, TF, OL, UOL, MCS, MCM, OE}\}$ is the per-answer type tag used by the judge, and o_i is the (possibly empty) list of multiple-choice options. The model f_θ (Qwen3-4B-Thinking-2507) is fixed; our prediction is $\hat{y}_i = E(f_\theta(P(x_i)))$, where P is a prompt template and E extracts the contents of the final `\boxed{}` group after stripping any `<think>...</think>` prefix. The objective is unified accuracy

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\mathrm{Judge}(\hat{y}_i, y_i, t_i, o_i)]\,,$$

where Judge is the type-aware symbolic equality judge from the competition's evaluation code. For prompting there is no training loss — we choose P to maximize Acc. For SFT we minimize causal-LM cross-entropy on a curated corpus $\mathcal{D}_{\mathrm{SFT}}$ of (problem, reasoning, answer) triples, with the loss applied only to response tokens (completion-only masking): the system prompt and question contribute no gradient, so the model learns to *produce* good reasoning and grader-aligned answers rather than to predict its own prompts.

Idea Summary: Our plan went in order of increasing cost: prompt engineering first, then LoRA SFT. We started with prompting because the starter prompt is generic — it tells the model where to put the answer but not what form it should take, and under a literal-comparison grader, surface-form mismatches alone (e.g. `1.363` vs. `atan(4.76)`) cost real points. Our format-aware prompt directly targets this: it mandates exact symbolic forms over decimal evaluations, full unrounded precision when decimals are required, disambiguated rounding rules, and a constrained reasoning style that suppresses runaway self-doubt loops; the MCQ variant adds a solve-then-match procedure to mitigate options-anchoring. The SFT stage then aimed to make these gains prompt-independent by baking the same formatting conventions and reasoning patterns into the weights: we distilled correct-only reasoning traces from a format-compatible teacher (Qwen3-30B-A3B-Thinking-2507) via rejection sampling and trained LoRA adapters on them, with a GRPO stage planned as a stretch goal contingent on a strong SFT checkpoint. In the end, prompting delivered the gains while SFT degraded performance, so our final submission relies on prompting alone.

Method Description: Prompt engineering effort went into two task-specific system prompts, one for free-response math problems and one for multiple-choice questions, motivated by the two dominant failure modes we observed in preliminary runs: answer-formatting mismatches and unproductive long reasoning cycles. Because the grader performs a literal string comparison, a mathematically correct answer in the wrong form (e.g., a rounded decimal where an exact expression

is expected) is scored as incorrect; our free-response prompt therefore specifies explicit formatting rules — preferring exact symbolic forms such as `atan(4.76)` or `(1/2)^(36/31)` over decimals, requiring full unrounded precision when a decimal is unavoidable, and enforcing problem-specific rounding conventions — and reinforces these rules with worked examples of correctly formatted final answers, effectively serving as few-shot demonstrations of the output format rather than of the reasoning itself. To curb overthinking, both prompts instruct the model to solve the problem directly with a single committed method, to perform exactly one brief sanity check rather than repeated re-derivations, and to treat long exploratory reasoning as a signal of being off-track; the prompt frames the model as a "confident expert" to discourage second-guessing loops. Finally, the multiple-choice prompt instructs the model to solve the problem *before* reading the options and forbids selecting the closest option when no exact match exists (outputting `NONE` instead), which guards against anchoring on distractor choices and against collapsing symbolic answers to nearby decimals.

Complementing the prompt-engineering approach, we explored supervised finetuning (SFT) with the goal of baking the formatting conventions and successful reasoning patterns directly into the model weights, so that they would survive distribution shift rather than depending entirely on the inference-time prompt. Since the competition prohibits external tools at inference time but does not prohibit external data generation beforehand, we distilled training data from a stronger teacher in the same model family, Qwen3-30B-A3B-Thinking-2507. The teacher was chosen deliberately: it shares the student's `<think>` output format, so its reasoning traces transfer to the student without any format conversion. For each training question of the 999 questions training set, we sampled $K = 5$ teacher traces, scored each with the competition judge, and retained up to two correct traces per question as training examples — a rejection-sampling scheme that ensures the student only imitates demonstrably successful reasoning. We then finetuned the 4B student model with LoRA, training low-rank adapters on top of the frozen base model, with the training prompt format chosen to match our inference prompt exactly to avoid any train–inference mismatch; hyperparameters are detailed in the Implementations section. We had planned a GRPO stage as a stretch goal on top of a solid SFT checkpoint, but because SFT ultimately degraded performance relative to prompt engineering alone and did not produce a checkpoint worth building on, we did not pursue it; we discuss this direction in Future Work.

4 Experiments

4.1 Compared configurations

We compare three configurations, all using the same fixed base model (Qwen3-4B-Thinking-2507) and the same vLLM inference stack, scored locally with the competition's symbolic judge on the same validation set:

- **Starter pipeline (default prompt):** The unmodified starter code: a brief generic system prompt that tells the model to put its final answer inside `\boxed{}`, sampled at Qwen's recommended thinking-mode defaults (`temperature=0.6`, `top_p=0.95`, `top_k=20`), with the last-`\boxed{}` extractor stripping the `<think>...</think>` prefix before parsing. This is our primary reference point.

- **Format-aware prompt (ours):** Same model, same sampling settings, same extractor — the only change is the system prompt. For free-form questions, the new prompt tells the model to keep answers in exact symbolic form rather than decimals (`atan(4.76)` instead of `1.363`, `5/8` instead of `0.625`), to give full unrounded precision when a decimal is actually required, and to follow the dataset's rounding conventions; it also constrains the reasoning style and includes seven worked examples covering the main answer-format families. The MCQ version tells the model to solve the problem first and only then look at the options, and explicitly forbids picking the closest option when the computed answer doesn't match any choice.

- **LoRA fine-tune (ours):** The format-aware prompt system with LoRA adapters trained on the distilled corpus described in Section 3, served through vLLM with adapter support.

4.2 Evaluation

We evaluate by running the competition's symbolic judge on our model's predictions and comparing them against the ground-truth labels in the validation set. We report overall accuracy and also split it into MCQ and free-form accuracy, because the two question types behave very differently — in our baseline they differ by 26 points — and a change that helps one can quietly hurt the other. Looking at both lets us catch problems that the overall number would hide. One practical lesson reinforced this choice: an early local grader we wrote ourselves reported substantially lower accuracy than the official judge, so we standardized all evaluation on the competition's own `judger` code (sympy-based symbolic equivalence with numerical tolerance and per-part list matching) to guarantee apples-to-apples comparisons between configurations.

4.3 Implementation details

Hardware. We run everything on Google Colab using an NVIDIA A100 (high-RAM) instance (Figure 1).

```
!nvidia-smi

Mon May 11 23:21:29 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-80GB          Off |   00000000:00:05.0 Off |                    0 |
| N/A   34C    P0             53W /  400W |       0MiB /  81920MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Figure 1: Output of `nvidia-smi` on our Colab Pro runtime: a single NVIDIA A100-SXM4-80GB (driver 580.82.07, CUDA 13.0). All training and evaluation runs in this report were executed on this hardware.

Inference setup. We serve Qwen3-4B-Thinking-2507 with vLLM 0.20 in `bfloat16`; the full engine and sampling configuration is shown in Figure 2. The engine is configured with `max_model_len=32768`, `max_num_seqs=64`, `max_num_batched_tokens=8192`, prefix caching enabled, and `gpu_memory_utilization=0.85`. For sampling we use Qwen's recommended thinking-mode defaults: `temperature=0.6`, `top_p=0.95`, `top_k=20`, `min_p=0.0`, with `max_tokens=24768`, which leaves roughly 8k tokens of context for the system prompt and question. These settings are identical across all three configurations, so accuracy differences come from the prompt and the adapter, not from decoding. For adapter inference, vLLM's LoRA support requires the maximum adapter rank at engine construction; we set `max_lora_rank=64` to cover our configurations.

Fine-tuning setup. We fine-tuned with LoRA adapters using TRL's `SFTTrainer`, keeping the base model's weights frozen and training only the low-rank adapter matrices. Hyperparameters (Figure 3): LoRA rank 32, $\alpha = 64$, dropout 0.05, no bias terms, adapters injected into all attention projections (`q/k/v/o_proj`) and MLP projections (`gate/up/down_proj`), 3 epochs, batch size 8, learning rate 1×10^{-4}, max sequence length 16384, with a short warmup. Loss was computed with completion-only masking, so the system prompt and question tokens are excluded from the objective. The training chat template and system prompt were made to match the inference-time prompt exactly — an earlier version of our pipeline appended inference-time instructions the model never saw during training, which silently hurt results until we caught it.

Training-data pipeline and data-quality fixes. The distilled corpus went through several iterations before it was trainable, and inspecting the raw examples (not just the loss curves) was what surfaced each problem. Three issues are worth documenting. First, an early version of the corpus had malformed thinking tags — every example contained a dangling `</think>` closing tag with no

```python
import sys, os

os.environ['HF_HOME'] = '/content/drive/MyDrive/hf_cache'
sys.stdout.fileno = lambda: 1
sys.stderr.fileno = lambda: 2

MODEL_ID = "Qwen/Qwen3-4B-Thinking-2507"

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

llm = LLM(
    model=MODEL_ID,
    dtype="bfloat16",                  # use "float16" if on T4
    enable_prefix_caching=True,
    gpu_memory_utilization=0.85,
    max_model_len=32768,               # lower if OOM
    trust_remote_code=True,
    max_num_seqs=64,
    max_num_batched_tokens=8192,
)

sampling_params = SamplingParams(
    max_tokens=24768,                  # leaves 8K for the input prompt
    temperature=0.6,
    top_p=0.95,
    top_k=20,
    min_p=0.0,
    presence_penalty=0.0,
    repetition_penalty=1.0,
)
```

Figure 2: vLLM engine and sampling configuration used for all inference runs. The Hugging Face cache is redirected to Google Drive so the downloaded model survives Colab session resets; `max_tokens=24768` leaves ∼8k tokens of headroom for the input prompt within the 32k context window.

```python
peft_config = LoraConfig(
    r=32,
    lora_alpha=64,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
)
```

Figure 3: PEFT `LoraConfig` used for fine-tuning: rank 32, $\alpha = 64$, dropout 0.05, no bias terms, with adapters injected into all attention projections (`q/k/v/o_proj`) and all MLP projections (`gate/up/down_proj`).

opening tag — which we fixed with a reconstruction pass over the assistant text. Second, we discovered answer leakage on MCQ examples: the gold answer had leaked into some training prompts, and the symptom downstream was that roughly half of the model's wrong MCQ answers boxed a computed *value* instead of an option *letter*. Third, we initially conflated answer formats: only single-letter boxed answers are true MCQ, while multi-letter boxed answers (e.g. `\boxed{G, B}`) behave as free-response under the judge, and our preprocessing had to respect that distinction. A separate operational pitfall: re-running the trainer without reloading the base model silently *stacks* LoRA adapters on top of each other and corrupts results, so we reloaded the base model between every run.

How we picked these settings. We did not sweep sampling parameters; we used the values Qwen publishes for the Thinking variant because those defaults are themselves the result of tuning by the model authors, and each evaluation already takes hours on a single A100. For LoRA we likewise used widely-recommended defaults (rank 32, $\alpha = 2r$) rather than sweeping, since our compute budget allowed only a small number of full fine-tuning runs.

Wall-clock cost. A baseline pass over the validation set took about two hours; the format-aware run took about four hours, because the longer system prefix and the more careful, fully-precise answers lengthen the output. Teacher-trace generation, training, and adapter evaluation each added hours on top of that, which is why the number of fine-tuning iterations we could afford was small.

4.4 Results

Baseline: The unmodified starter pipeline reached **61.90%** overall, with **79.47%** on multiple-choice and **53.13%** on free-form. The 26-point gap between the two question types told us where the headroom was: multiple-choice was already pretty strong, but more than half of the free-form questions were being marked wrong.

Why free-form was failing: We read through a sample of the wrong free-form answers and found a clear pattern. The model was often doing the math correctly, but writing the answer in a form the grader wouldn't accept — `1.363` when the grader wanted `atan(4.76)`, `0.625` when it wanted `5/8`, or `105950` when it wanted the unevaluated form `325*(1+325)`. We also saw a separate failure mode where the model would get stuck in a self-doubt loop and produce a very long response that never committed to a final answer at all.

Experiment 1 — Format-aware prompting: On the validation set, the format-aware prompts reach **67.37%** overall, with **82.62%** on MCQ and **59.91%** on free-form (Table 1). The improvement is bigger on free-form ($\sim$+6.8 points) than on MCQ ($\sim$+3.2 points), which is exactly what our error analysis predicted: free-form was where the format-related losses were concentrated, and the new prompt is mostly aimed at format. The MCQ gain probably comes from the solve-then-match instruction rather than the format rules. The MCQ-vs-free-form gap shrank from 26 to about 23 points but remained large, which is what motivated fine-tuning next.

Table 1: Baseline (starter prompt) and our format-aware prompt, both run on the same validation set and scored locally with the competition's symbolic judge.

	MCQ	Free-form	Overall
Baseline	79.47%	53.13%	61.90%
Format-aware prompt	82.62%	59.91%	67.37%
Δ	+3.15	+6.78	+5.47

Experiment 2 — LoRA fine-tuning: Our hypothesis was that the model could reason, but did not reliably produce the right format and final answer, and that fine-tuning on correct, grader-verified reasoning traces would bake the desired behavior into the weights. The result went the other way: the fine-tuned model scored **64.29%** overall (76.69% MCQ, 58.04% free-form), a drop of about 3 points relative to the prompt-only system, with the largest regression on MCQ (Table 2).

Table 2: Prompt-only system versus LoRA fine-tune, same validation set, same judge.

	MCQ	Free-form	Overall
Format-aware prompt	82.62%	59.91%	67.37%
LoRA fine-tune	76.69%	58.04%	64.29%
Δ	−5.93	−1.87	−3.08

Why fine-tuning underperformed: Two factors stand out. First, the corpus was small: after rejection sampling, we had high-quality traces for only $\sim$950 problems, which is little signal against a 4B model that is already strong at this task. Second — and we believe more important — many of the teacher's reasoning traces were noticeably *shorter* than the base model's natural thinking-mode output. Training on them appears to have taught the model to under-reason: post-fine-tune outputs committed to answers earlier and thought less, which hurts exactly the harder problems where extended reasoning is the base model's strength. This effect was invisible in the loss curves and only became obvious when we inspected generated examples side by side. Given a degraded checkpoint and no compute budget for a substantially larger corpus, the right engineering decision was to keep the base model with format-aware prompting as our final submission rather than ship a worse fine-tune.

Leaderboard result and generalization: Our final submission — the base model with format-aware prompts — placed **12th** on the final leaderboard, an improvement of **36 positions** over our

original public-leaderboard standing. The gains transferred, but not perfectly: our validation accuracy was somewhat higher than our held-out test accuracy, a reminder that a validation set carved from the public data is still closer in distribution to the public data than the hidden test set is, and that it is easy to overfit prompt-design decisions to the data you can see.

A note on noise: All runs use `temperature=0.6` sampling, so individual numbers shift by about a percentage point on re-runs. The prompting gain (+5.47) and the fine-tuning regression (-3.08) are both larger than this, so neither is sampling noise, but we did not compute formal confidence intervals.

4.5 Ablations

Our experimental design changes exactly one component per comparison, so our two main results double as controlled ablations.

Prompt ablation. The baseline and format-aware runs differ *only* in the system prompt: same model, same vLLM engine settings, same sampling parameters, same answer extractor. Removing the format-aware prompt (i.e., reverting to the starter prompt) costs 5.47 points overall (-3.15 MCQ, -6.78 free-form), so the prompt accounts for the entirety of our improvement over the starter pipeline. The per-type split further isolates *which* prompt matters where: the free-form prompt's formatting rules and worked examples drive the larger free-form gain, while the MCQ prompt's solve-then-match procedure drives the smaller MCQ gain — consistent with our error analysis, which found format mismatches concentrated in free-form answers and options-anchoring in MCQ.

Adapter ablation. The LoRA configuration adds exactly one component — the trained adapter — on top of the format-aware system, holding the prompt, decoding settings, and judge fixed. Adding the adapter costs 3.08 points overall (-5.93 MCQ, -1.87 free-form), which cleanly attributes the regression to the fine-tuning itself rather than to any other pipeline change. The fact that MCQ regresses nearly twice as much as free-form is itself diagnostic: the training corpus was dominated by free-form-style reasoning traces, and the under-reasoning behavior it induced hurt the question type that depends most on careful elimination of distractor options.

What we did not ablate. We did not ablate individual rules *within* the free-form prompt (the symbolic-form preference, the rounding rules, the seven worked examples, and the anti-overthinking instructions), because each validation pass takes 2–4 hours on a single A100 and a per-rule sweep would have consumed our remaining compute budget. Likewise, we could not afford to ablate LoRA hyperparameters (rank, learning rate, epochs) across multiple training runs. Identifying which prompt components carry the most weight is the cheapest high-value experiment we would run with more compute.

5 Discussion

Achievements. Three concrete things came out of this project. First, a reproducible baseline with per-type diagnostics: 61.90% overall, with the 26-point MCQ/free-form gap serving as the diagnostic that directed all subsequent work. Second, a +5.47-point improvement from format-aware prompting at zero training cost, concentrated exactly where the error analysis predicted, which carried through to a 12th-place finish and a 36-position climb on the leaderboard. Third, a complete, working parameter-efficient fine-tuning pipeline — teacher-model trace generation with rejection sampling, data cleaning, LoRA training with completion-only masking, and adapter-based vLLM evaluation — which, although it produced a negative result this time, is the infrastructure a successful fine-tune would be built on.

The negative result. Fine-tuning made our system worse, and we think it is more useful to say that plainly than to bury it. The diagnosis matters more than the number: the failure was not "LoRA doesn't work" but "our training data taught the model a bad habit." Short teacher traces suppressed the base model's thinking, and a $\sim$950-problem corpus was too little signal to teach much else. The decision this enabled — staying with the base model rather than shipping a degraded fine-tune — is itself a result: knowing *when not to deploy* a trained model is part of doing this well.

Lessons learned. (1) *Quality beats quantity for SFT data, and trace properties matter as much as correctness.* A correct-but-short trace is still a harmful training example for a thinking model. (2) *Inspect the data, not just the metrics.* Every serious data problem we hit — malformed thinking tags, answer leakage, short traces — was invisible in loss curves and obvious within minutes of reading raw examples. (3) *Match training and inference prompts exactly.* A silent mismatch between the two cost us a debugging cycle. (4) *Standardize on the official judge.* Our hand-rolled grader disagreed with the official one and would have led us to wrong conclusions. (5) *Per-type diagnostics catch what aggregates hide.* The MCQ regression after fine-tuning (-5.93) was much larger than the overall regression suggested. (6) *Improvements don't automatically generalize.* Validation gains shrank on the hidden test set, so guarding against overfitting to visible data has to be deliberate.

Bottlenecks and Mitigation. The biggest bottleneck throughout was compute time. A full validation pass takes two to four hours, and the fine-tuning loop (generate teacher traces, clean, train, evaluate) takes most of a day end to end, which put a hard cap on the number of fine-tuning iterations we could attempt. Two pieces of that loop were especially expensive: training itself — LoRA runs over 16k-token reasoning traces on a single GPU each consumed a large fraction of a Colab session, capping how many configurations we could try — and data generation, since producing our own reasoning-trace corpus with a teacher model meant hours of sampling for every new dataset version. We mitigated this by upgrading to Colab Pro A100 high-RAM instances, by evaluating on a fixed held-out validation set rather than the full public set, and by keeping the SFT corpus small enough to train in a single session. The other bottleneck was the fragility of the training stack on Colab — stale kernel state, library version mismatches, and the adapter-stacking pitfall — which we handled with strict run-order discipline (always reload the base model before training).

Team Responsibilities. Our team has three members:

- **Samson** — strong background in machine learning and optimization. Set up the baseline pipeline on Colab, tuned the inference parameters, and worked on teacher-trace generation for the SFT corpus.
- **Edward** — strong background in optimization. Designed and tested the format-aware system prompts and led the error analysis that drove them.
- **Junior** — background in writing parallel and performance-oriented code. Focused on inference efficiency, the vLLM stack, and the LoRA training and adapter-evaluation pipeline, including the data-quality debugging.

All three of us worked on the writeup together.

6 Future Work

With more compute and time, the directions we would pursue, roughly in order of expected value:

- **A larger, higher-quality SFT corpus with full-length reasoning traces.** Our experiments suggest trace length and quality matter more than raw quantity; the fix is to filter teacher traces by length and reasoning style, not just correctness, and to scale the corpus well beyond $\sim$1k problems.
- **LoRA hyperparameter sweeps** (rank, learning rate, target modules, epochs) instead of relying on reasonable defaults — with our compute budget, the rank-32 configuration was effectively a single shot.
- **A full GRPO run with verifiable-answer rewards** on top of a solid SFT checkpoint — the natural next step for a task with automatically checkable answers, and the step we never reached because SFT did not produce a checkpoint worth building on.
- **Self-consistency / majority voting and best-of-N sampling** at inference time, as training-free ways to trade compute for accuracy on top of the format-aware prompt.
- **More systematic error analysis and held-out evaluation,** to catch overfitting to the public data earlier and keep targeting the highest-impact failure modes.

LLM Usage

We used a general-purpose LLM assistant to proofread this report, to help with phrasing and clarity, and as a debugging aid during development. All technical decisions, experiments, code, and final wording are our own, and we take full responsibility for the contents of this report.

References

[1] Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[2] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35:22199–22213, 2022.

[4] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.